Архитектура решения

Цифровой кредитный конвейер для малого и микробизнеса

Alfa Hack 2026

1 Общая схема архитектуры

Решение построено по классической клиент-серверной модели с изоляцией ML-слоя. Взаимодействие между компонентами происходит через REST API. Все части разворачиваются в изолированных Docker-контейнерах и связываются через `docker-compose`.

1.1 Технологический стек

Слой	Технологии
Frontend	React 18, TypeScript, Vite, Tailwind CSS, Framer Motion
Backend	Python 3.11, FastAPI, SQLAlchemy, Uvicorn
База данных	SQLite (MVP), спроектирована под PostgreSQL
ML-слой	CatBoost, Scikit-learn, Pandas, SHAP
Моки интеграций	FastAPI-сервис, имитирующий ЕГРЮЛ, ФНС, БКИ
Инфраструктура	Docker, Docker Compose

2 Компоненты системы

2.1 Frontend (клиент + сотрудник)

Одностраничное приложение (SPA) на React. Реализовано два ролевых интерфейса:

- **Кабинет клиента:** дашборд, кредитный калькулятор, форма заявки, документы, уведомления, экран подписания договора.
- **Рабочий стол сотрудника:** таблица заявок с поиском и фильтрацией, детальная карточка заявки, панель анализа рисков (SHAP-факторы), интерфейс принятия решений.

2.2 Backend (FastAPI)

Серверное приложение, реализующее:

- CRUD-операции с заявками и документами.
- Фоновый конвейер обработки заявки через `BackgroundTasks`.
- Обогащение данных через моки внешних источников.
- Вызов ML-моделей и движка решений.
- Разделение данных на `serialize_for_client` и `serialize_for_employee` для защиты приватной информации (PD, SHAP-факторы).

2.3 ML-слой

- **Рекомендательная модель:** правила фильтрации каталога + логистическая регрессия для сортировки продуктов.
- **Скоринговая модель:** CatBoost, обученный на синтетических данных, выдающий вероятность дефолта (PD) и SHAP-значения.
- **Движок решений:** набор правил поверх PD, суммы и стоп-факторов, формирующий итоговый статус заявки.

2.4 Моки внешних систем

Согласно требованиям кейса, реальные интеграции с ЕГРЮЛ, ФНС и БКИ запрещены. Вместо них используется локальный FastAPI-сервис на порту `8001`, который возвращает синтетические JSON-ответы по ИНН:

- `/egrul/{inn}` — статус компании, дата регистрации, ОКВЭД.
- `/fns/{inn}` — наличие налоговой задолженности.
- `/bki/{inn}?consent=true` — кредитная история, просрочки, запросы.

3 Логика обработки заявки (Pipeline)

Процесс полностью автоматизирован и запускается при вызове `POST /applications/{id}/submit`. Каждый этап сопровождается сменой статуса и записью в историю (`status_history`).

1. **Проверка данных** (`checking`). Запросы к мокам ЕГРЮЛ, ФНС, БКИ с таймаутом 3 секунды и 1 ретраем. При недоступности БКИ — статус `manual_review`.
2. **Обогащение.** Расчет возраста бизнеса по дате регистрации из ЕГРЮЛ. Сохранение ответов моков в БД.
3. **Оценка риска** (`scoring`). Формирование вектора из 15 признаков и вызов `predict_with_explanatio` Получение PD и топ-5 SHAP-факторов.
4. **Решение** (вызов `decide()`). Применение кредитной политики.
5. **Смена статуса.** Заявка переходит в один из финальных статусов.

3.1 Правила Движка Решений

Условие	Решение
PD $< 5\%$ и сумма ≤ 3 млн	Автоматическое одобрение
PD $5\% - 15\%$	Ручная проверка
PD $> 15\%$	Автоматический отказ
Сумма > 3 млн	Ручная проверка (даже при низком PD)
Просрочка > 90 дней, банкротство, налог. долг	Отказ по стоп-фактору
Отсутствие обязательных данных	Статус `need_info`

Конфигурируемость

Все пороги вынесены в файл `scoring/decision/config.json`. Для изменения кредитной политики достаточно отредактировать JSON и перезапустить контейнер — код не меняется.

4 Модель данных

Таблицы в SQLite (схема совместима с PostgreSQL):

- **users:** клиенты и сотрудники (роли).
- **applications:** заявки со всеми 15 ML-признаками, JSON-полями для ответов моков, PD, SHAP-факторов и итогового решения.
- **status_history:** аудит смены статусов (кто, когда, комментарий).

5 Безопасность и Enterprise-подход

- **Разделение данных:** клиент не получает PD и SHAP-факторы, только понятные причины решений.
- **Аудит:** каждая смена статуса логируется с указанием автора (клиент / система / сотрудник).
- **Версионирование моделей:** в каждом решении сохраняется `model_version` и `rules_version` для воспроизводимости.
- **Отказоустойчивость:** при недоступности внешних источников заявка не падает с ошибкой, а уходит на ручную проверку.
- **Идемпотентность:** нельзя отправить заявку из статуса, отличного от `draft` или `need_info`.

6 Развертывание

Все компоненты запускаются одной командой:

```
docker compose up -build
```

После запуска доступны:

- Frontend: `http://localhost:5173`
- Backend API: `http://localhost:8000`
- Swagger: `http://localhost:8000/docs`
- Моки: `http://localhost:8001`